

Intelligent Home Search System: Hybrid Retrieval for California Listings

Joshua (Zhecheng) Ren

IDX Exchange, California, USA

joshua-ren@berkeley.edu

Abstract

Property search combines exact database constraints with preferences that are expressed in ordinary language. Treating every term as a filter risks suppressing suitable listings; treating the request as free text risks violating explicit requirements. This report describes an intelligent home search system for active California listings that separates these two roles. The system constructs an immutable public snapshot from MLS records, screens listing remarks before publication, parses requests into hard filters and soft signals, and retrieves candidates through lexical, dense, and listing-signal indexes. Reciprocal-rank fusion combines the candidate sources, while an optional Cross Encoder reranks a bounded window. Three execution profiles expose distinct cost-quality trade-offs without changing the publication boundary. On a frozen 12-query relevance test, the quality profile reaches Precision@5 of 0.867, NDCG@5 of 0.901, and MRR@5 of 0.917. The implementation is packaged as a FastAPI service with a Streamlit product interface, Redis-backed caching, and versioned runtime artifacts.

Index Terms: real estate search, hybrid retrieval, structured query understanding, reciprocal-rank fusion, Cross Encoder reranking

1. Introduction

Residential search is neither a conventional faceted search task nor a purely semantic retrieval task. A request such as “three bedrooms in Irvine under \$900k with a backyard” contains non-negotiable eligibility constraints and a preference that may be described differently in listing text. The system must honor the numerical and geographic constraints exactly, while still recognizing related language such as “outdoor living” or “private yard.” This distinction is especially important in MLS data, where trusted structured fields and long, inconsistently written remarks coexist.

The system described here uses a retrieval architecture that preserves this distinction. Structured city, price, bedroom, bathroom, and square-footage requirements are applied before retrieval. Remaining terms are represented as soft signals and contribute evidence during ranking. Lexical BM25 retrieval [1] remains useful for specific terminology; dense sentence representations improve recall for paraphrases [2, 3]; and a dedicated signal index preserves the category-aware evidence extracted from listing text. These sources are fused with reciprocal-rank fusion (RRF) [4], then optionally reranked by a Cross Encoder [5].

The design is intended as an operational search service rather than an isolated ranking experiment. It therefore includes a publication boundary, immutable artifact validation, bounded reranking, fallback behavior, caching, readiness checks, and separate public and administrator interfaces. The following sections describe the data and artifact boundary, the retrieval path, and the frozen evaluation used to select the final configuration.

2. Datasets

The production snapshot is built from the California `rets.property` MLS table. At the measured build, it contained 53,122 records with an address, city, price, bed and bath counts, square footage, and a listing remark. Raw MLS exports remain local and are not versioned. The snapshot build retained 53,091 records with a `pass` compliance status; 52,763 of those records had usable text for lexical or dense retrieval. Listings with insufficient text remain eligible for structured fallback when the request contains no meaningful soft requirement.

Evaluation assets are deliberately separated from the source table. Query parsing uses 120 labeled search requests. Listing-signal quality uses 200 reviewed listings, and intent classification uses 72 held-out requests. The end-to-end relevance evaluation uses 12 held-out queries and a frozen set of 849 graded query-listing judgments. Any candidates returned but not previously judged were labeled in a blinded delta pool before the final test was frozen. No retrieval or reranking configuration was changed after that freeze.

3. System Description

3.1. Pass-Only Snapshot Construction

The offline build establishes the system’s publication boundary. Each source record is checked by the versioned `federal-1.1` policy. Findings are deduplicated by severity, span length, and text position; an error produces `blocked`, a warning produces `review`, and no finding produces `pass`. Only `pass` records enter the public catalog, summary artifact, and retrieval indexes. The checker is a deterministic screening control, not a substitute for legal or policy review.

For every admitted record, the builder preserves trusted structured fields and stores both the original remark and a normalized version. It extracts canonical text signals, produces a concise fact-first summary, and builds three retrieval artifacts: a BM25 index, a dense index, and a signal index. The snapshot manifest records the source, public, and retrievable counts; policy version; dense model; and SHA-256 checksums for each artifact. Validation requires catalog, audit, summary, lexical, dense, and signal IDs to agree before an atomic active-snapshot pointer is updated. This makes a partially generated artifact unavailable rather than silently serving mixed versions.

3.2. Query Interpretation and Structured Eligibility

The query parser combines normalization, city vocabulary matching, numerical patterns, and a real-estate taxonomy. It emits a flat parse and a split representation. The hard portion contains only fields supported by the database predicate path: city, price, bedrooms, bathrooms, square footage, and sort direction. Property type, amenities, rooms, interior and exterior features, location features, condition terms, and negative preferences remain soft signals. This prevents weak or incomplete

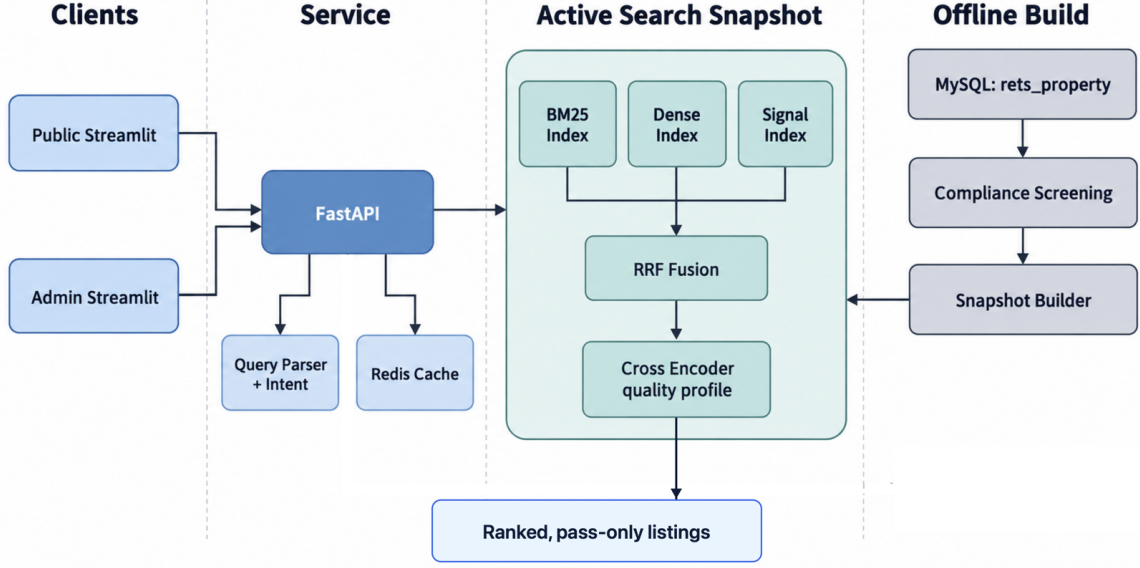


Figure 1: System architecture. Offline construction turns MLS records into a validated, compliance-screened search snapshot. At request time, FastAPI applies query understanding, structured eligibility, retrieval, and optional reranking without rereading raw listing remarks.

mention-level evidence from excluding records before ranking.

Hard predicates are translated into parameterized MySQL queries. Their result set is intersected with the active snapshot’s `pass` IDs, so a record must satisfy both database eligibility and the publication boundary. The parser also uses the calibrated query-intent classifier as an auxiliary language signal. An answerability check runs before retrieval and returns a clear response when the request cannot be served from the available listing fields.

3.3. Hybrid Candidate Retrieval and Fusion

After eligibility, the service retrieves candidates from independent sources. BM25 contributes exact lexical matches. The dense retriever embeds the query with `all-MiniLM-L6-v2`, following the bi-encoder retrieval pattern enabled by Sentence-BERT [2]; it returns nearest listing vectors among eligible records. The signal index matches normalized, bucketed listing signals such as *pool*, *open floor plan*, and *near parks*. Signal normalization resolves aliases and category conflicts, removes generic values, and promotes specific values over general ones.

The default candidate limits are 100 dense results, 100 BM25 results, and up to 200 signal results. Dense, BM25, and signal work is independent after hard filtering and executes through a two-worker retrieval pool. Candidate IDs are unioned and deduplicated. Text-source ranks are combined using

$$\text{RRF}(d) = \sum_{s \in S(d)} \frac{1}{k + r_s(d)}, \quad k = 60, \quad (1)$$

where $S(d)$ is the set of text retrievers returning listing d and $r_s(d)$ is its rank in source s . Selected signal matches add a fixed, match-strength-dependent boost; explicit negative matches receive a small penalty. Thus, signals influence ranking without becoming broad SQL predicates.

3.4. Search Profiles, Reranking, and Degradation

The public interface exposes three stable profiles. `fast` uses BM25 after hard filtering. `balanced` combines dense, BM25, and signal candidates with the fusion procedure above. `quality`, the default, reranks the first 50 fused candidates with `cross-encoder/ms-marco-MiniLM-L6-v2`. Each reranker document combines trusted fields, up to 1,500 characters of normalized remarks, canonical signals, and a short generated summary. This gives the Cross Encoder structured context without sending an unbounded listing description to the model.

The quality reranker has a bounded semaphore because the CPU deployment cannot safely execute several Cross Encoders at once. A reranker exception falls back to the RRF order and is recorded in response metadata. A saturated quality queue returns an explicit retryable error rather than substituting a lower-quality profile invisibly. Independent retriever failures are likewise recorded as degraded components; remaining sources can still produce a response. Price sorts follow a separate contract: pure price requests use stable structured ordering, while price requests with preferences first prioritize complete and partial signal matches before applying price order.

3.5. Result Formation and Structured Fallback

Ranking operates on listing IDs; presentation is assembled only after the final order is known. For each returned ID, the service joins the public catalog fields, generated summary, matched and excluded signals, source ranks, and profile-specific rank metadata. The product interface can then show a concise result card immediately and obtain the original public remark through a batch details route when the user expands a listing. Internal model scores and experiment-only traces are not part of the pub-

Search Profile Execution Paths

All profiles operate on the same compliance-screened, hard-filtered snapshot.

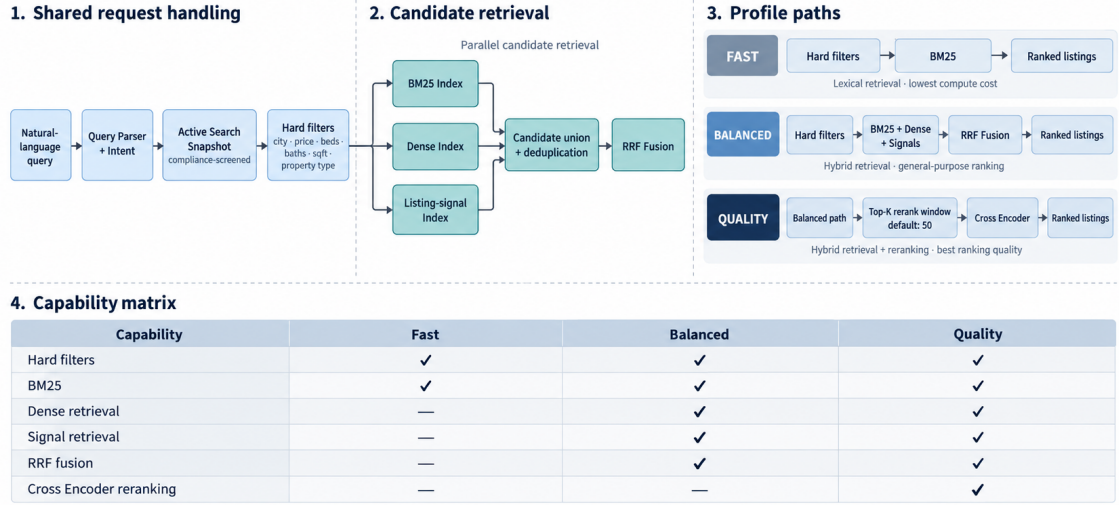


Figure 2: *Profile-specific execution paths.* All profiles share parsing, hard filtering, and the same compliance-screened snapshot. *fast* retains lexical retrieval only; *balanced* adds dense and signal candidates with RRF; *quality* adds bounded Cross Encoder reranking.

lic response contract.

The system also distinguishes an empty text-retrieval result from an empty eligible set. When a request contains only structured constraints, eligible listings without retrievable text can fill an otherwise short result set in stable ID order. This structured fallback is disabled when the user supplied a meaningful soft requirement: returning a textless listing in response to a request for a pool, view, or backyard would conceal the absence of preference evidence. The same principle guides price sorting. Pure price requests are ordered by trusted structured price, while price requests with preferences retain signal-aware tiers before the final price order.

3.6. Service and Product Boundary

FastAPI loads the active snapshot and shared components once during startup. Readiness requires Redis, the active snapshot, the intent artifact, the dense model, and the Cross Encoder to initialize. Search responses are cached in Redis with keys that include the active snapshot and requested profile, preventing stale results after activation. Public POST endpoints use a rolling per-IP request limit. The public Streamlit workspace exposes search, sorting, paging, listing details, and profile selection; the administrator workspace adds same-query profile comparison and aggregate latency and feedback metrics. In production, Caddy is the public HTTPS boundary and the API remains on the private service network.

4. Evaluation

Evaluation combines component checks with an end-to-end frozen relevance test. Parser evaluation reached exact hard-filter and soft-signal match rates of 1.000 on the 120-query set; the full parse match rate was 0.917 because it penalizes additional nonessential output. On the 200-listing signal set,

Table 1: *Frozen 12-query relevance evaluation through the deployed API.*

Profile	P@5	NDCG@5	MRR@5
Fast	0.667	0.566	0.794
Balanced	0.800	0.744	0.917
Quality	0.867	0.901	0.917

trusted structured-field accuracy was 1.000, free-text signal F1 was 0.799, and keyword integrity was 1.000. These component measurements verify that the signal evidence used by the hybrid path is grounded in the intended schema.

Table 1 reports the frozen relevance results reproduced through the deployed API. The profiles share the same explicit-filter behavior; the comparison isolates the ranking path. The quality profile improves both Precision@5 and NDCG@5 over the lexical and fused baselines. Its MRR matches the balanced profile, indicating that the primary gain on this small test set is stronger ordering within the top five rather than moving the first relevant result earlier.

Latency is profile-dependent and must be interpreted separately from relevance. Table 2 reports client-observed HTTP latency from local and deployed runs using the same initial/repeat request pattern. The local measurements characterize the development host; the deployed measurements include the VM and tunnel path, so they are not a direct hardware comparison. Serialized Cross Encoder inference nevertheless dominates quality-profile cache misses in both settings. These figures are release-validation measurements rather than an availability or latency SLA. They support the profile design: *fast* is appropriate where lexical speed is sufficient, while *quality* makes its additional cost explicit instead of hiding it behind a single ranking mode.

Table 2: Observed HTTP latency in milliseconds (40 requests per profile and cache state; values shown to two decimal places). Local runs target the development API; deployed runs target the VM through an SSH tunnel.

Profile	Cache hit			Cache miss		
	P50	P90	P95	P50	P90	P95
<i>Local development API</i>						
Fast	9.20	11.79	12.08	99.58	251.75	261.71
Balanced	10.27	12.51	12.87	114.22	416.81	455.68
Quality	10.70	12.90	13.06	1,047.56	1,299.88	1,338.35
<i>Deployed VM via SSH tunnel</i>						
Fast	67.04	73.86	76.63	162.20	547.76	552.90
Balanced	69.75	76.51	78.25	172.68	679.52	728.80
Quality	69.52	75.07	76.73	4,133.26	4,578.21	4,652.66

The relevance set is intentionally small and is best read as a frozen release gate, not a claim of market-wide ranking performance. It covers explicit constraints, lexical preferences, semantic preferences, and mixed requests, but it cannot substitute for a larger independently judged query log. The evaluation nevertheless exposes an important product-level property: all three profiles preserve the same hard-filter contract, so quality changes affect ranking rather than the eligible inventory. This makes profile comparisons interpretable and permits the service to trade compute for relevance without weakening explicit buyer requirements.

5. Conclusion

This report presented a production-oriented search system that separates strict listing eligibility from preference-sensitive ranking. The pass-only snapshot moves compliance screening and artifact validation off the request path; query interpretation keeps database-backed constraints distinct from soft signals; and hybrid retrieval combines lexical, semantic, and canonical feature evidence before bounded reranking. The resulting profiles provide a visible quality-latency trade-off while preserving the same catalog and hard-filter contract. The next practical improvements are a larger independently judged relevance set,

a stable production domain, and capacity planning for quality-profile reranking.

6. References

- [1] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.
- [3] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” *arXiv preprint arXiv:2002.10957*, 2020.
- [4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [5] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” *arXiv preprint arXiv:1901.04085*, 2019.